

Class And Objects

21 April 2026 05:38

In object oriented programming, classes and objects play a vital role in programming. These are the two main pillars of OOP (Object oriented programming) Without classes and objects we cannot create a program in Java.

What is a class in Java?

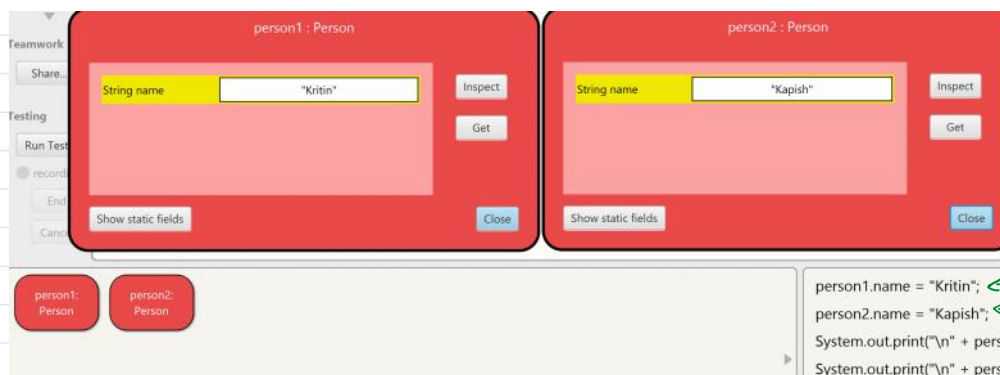
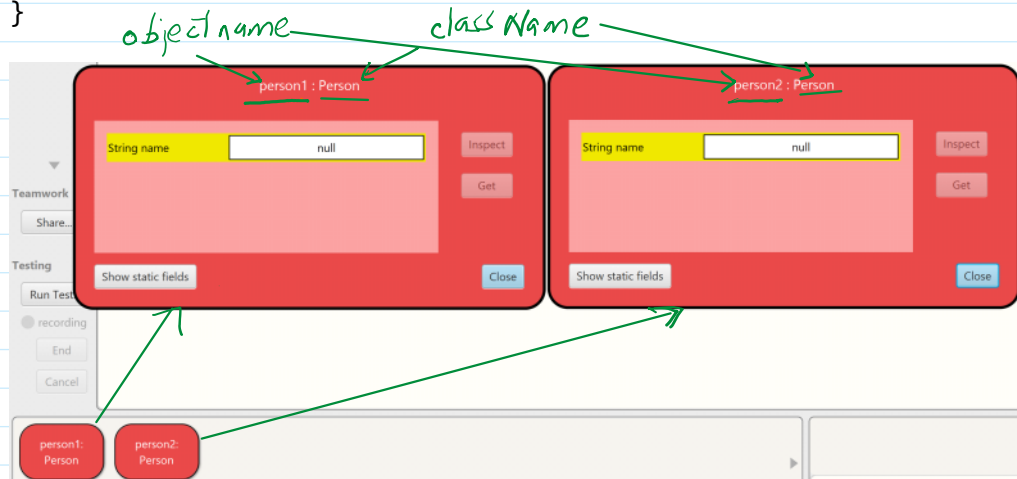
A class is a group of objects that have common properties. It is a template or blueprint from which objects are created. It is a logical entity, can't be physical.

A Java class contains:

- **Fields:** Variables stated inside a class that indicate the status of object formed from that class are called fields, sometimes referred to as instance variables. They specify the data that will be stored in each class object. Different access modifiers such as public, private, protected, and default can be applied to fields to regulate their visibility and usability.

For example:

```
public class Person
{
    String name; //field
}
```



- **Methods:** Methods or functions defined inside a class that include the actions or behavior that objects of that class are capable of performing. These techniques allow the outside world to function and change the objects state (fields) Additionally, method can be void (that is they return nothing) or have different access modifiers. They can also return values.

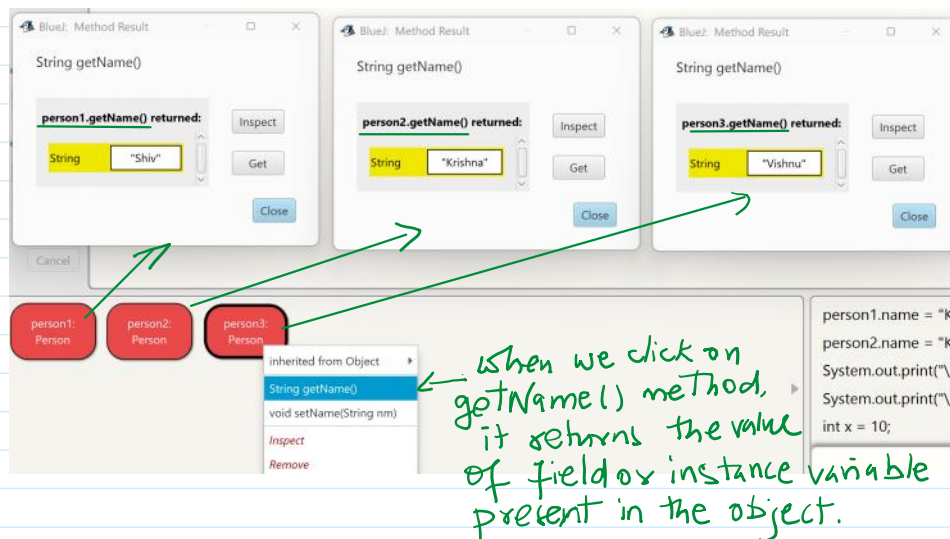
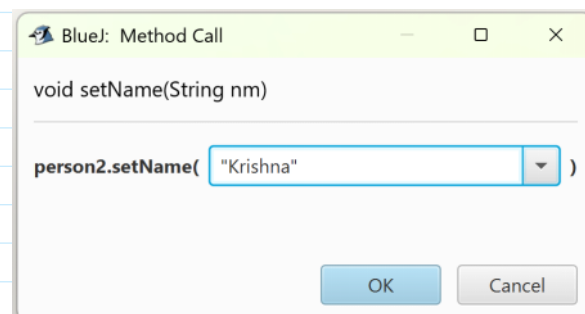
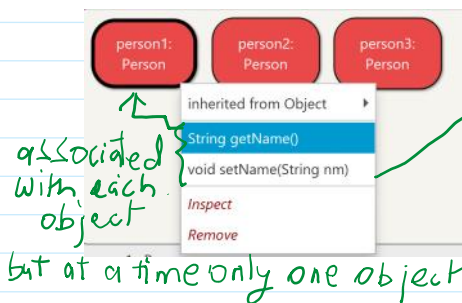
For example:

```
public class Person
{
    String name; //field

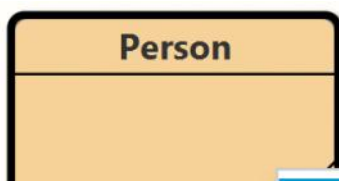
    void setName(String nm){
        name = nm;
    }

    String getName(){
        return name;
    }
}
```

after click on setName(String nm)
we get this input dialog box



- **Constructors:** Constructors are a unique methods that are used to initialize class object. When an object of the class is created using the new keyword, they are called with the same name of the class. Constructors can initialize the fields of an object or carry out any additional setup that's required when an object is created.



method name is same as class name.



new Person()

method name is same as class name.

method call is done by the 'new' keyword.

This is a constructor defined by JVM.

Scanner sc = new Scanner(System.in); ← in code form.
Object. keyword. constructor

The job of new keyword is allocate memory on the heap (a large memory area) for the object.

```
public class Person
{
    String name; //field

    Person(){ //Constructor
        name = null;
        System.out.print("\nObject created");
    }

    void setName(String nm){
        name = nm;
    }

    String getName(){
        return name;
    }
}
```

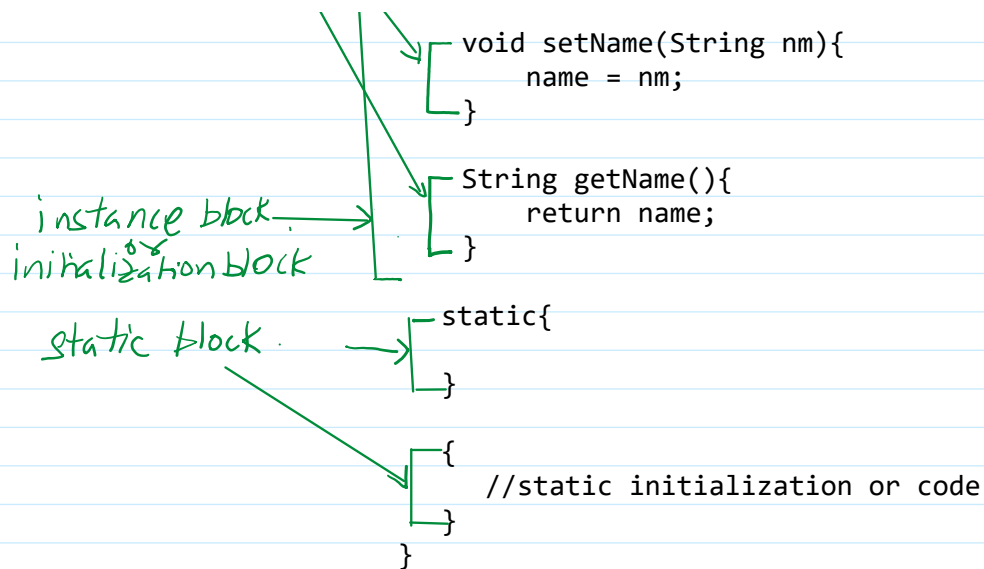
- **Blocks:** Within a class, Java allows two different kinds of blocks: instance blocks and commonly referred to as initialization blocks and static blocks. Static blocks, which are usually used for static initialization, are only executed once when the class is loaded into memory. Instance block can be used to initialize instance variable and are executed each time a class object is generated.

blocks →

```
public class Person
{
    String name; //field

    Person(){ //Constructor
        name = null;
        System.out.print("\nObject created");
    }

    void setName(String nm){
        name = nm;
    }
}
```



For example:

```
package Loops;
```

```

/**
 * Write a description of class Person here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Person
{
    String name; //field

    static String City;
    Person(){ //Constructor
        name = null;
        System.out.print("\nObject created");
    }

    void setName(String nm){
        name = nm;
    }

    String getName(){
        return name;
    }

    static{
        //System.out.print("\nThis is pure static block");
        City = "Kolkata";
        //This block is used to initialize the static variables only.
    }

    {
        System.out.print("\fThis is a person class");
        //This block will execute whenever we create object of the class.
    }
}

```

}

- **Nested class and interface:** Java permits the nesting of classes and interfaces inside other classes and interfaces. The members (fields, Methods) Of the enclosing class are accessible to the nested classes which can be static or non-static. Nested interfaces can be used to logically group related constants and methods together because they are implicitly static.